Hadoop Installation and Other Steps:

Either follow A (Install in Local System) or B (Run as a Docker Container in Local System)

A. Install in Local System

Please install Hadoop using the following link:
https://phoenixnap.com/kb/install-hadoop-ubuntu

B. Run as a Docker Container in Local System

1. We can try Hadoop docker. Link: https://hub.docker.com/r/apache/hadoop

2. Pull the hadoop image: sudo docker pull apache/hadoop:3

3. Create a basic yaml file (name docker-compose.yaml) like below:

```
version: "2"
services:
  namenode:
    image: apache/hadoop:3
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - 9870:9870
    env_file:
      - ./config
    environment:
        ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"
  datanode:
    image: apache/hadoop:3
    command: ["hdfs", "datanode"]
    env_file:
      - ./config
  resourcemanager:
    image: apache/hadoop:3
    hostname: resourcemanager
    command: ["yarn", "resourcemanager"]
    ports:
       - 8088:8088
    env_file:
      - ./config
    volumes:
      - ./test.sh:/opt/test.sh
```

```
  nodemanager:
    image: apache/hadoop:3
    command: ["yarn", "nodemanager"]
    env_file:
      - ./config
```

4. Create a config file (name config) like below:

```
CORE-SITE.XML_fs.default.name=hdfs://namenode
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1
MAPRED-SITE.XML_mapreduce.framework.name=yarn
MAPRED-SITE.XML_yarn.app.mapreduce.am.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
MAPRED-SITE.XML_mapreduce.map.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
MAPRED-SITE.XML_mapreduce.reduce.env=HADOOP_MAPRED_HOME=$HADOOP_HOME
YARN-SITE.XML_yarn.resourcemanager.hostname=resourcemanager
YARN-SITE.XML_yarn.nodemanager.pmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.delete.debug-delay-sec=600
YARN-SITE.XML_yarn.nodemanager.vmem-check-enabled=false
YARN-SITE.XML_yarn.nodemanager.aux-services=mapreduce_shuffle
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.maximum-applications=10000
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.maximum-am-resource-percent=0.1
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.resource-calculator=org.apache.hadoop.yarn.util.resource.DefaultResourceCalculator
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.queues=default
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.capacity=100
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.user-limit-factor=1
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.maximum-capacity=100
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.state=RUNNING
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.acl_submit_applications=*
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.root.default.acl_administer_queue=*
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.node-locality-delay=40
```

CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.queue-mappings=
CAPACITY-SCHEDULER.XML_yarn.scheduler.capacity.queue-mappings-ove
rride.enable=false

5. Run the docker containers using docker-compose: sudo docker-compose up -d

6. Login into a node: sudo docker exec -it test1_namenode_1 /bin/bash

7. Visualise the dashboard in your localhost:

   Access to localhost:9870 in your browser

8. Create input files
   mkdir input
   echo "Hello Docker" >input/file2.txt
   echo "Hello Hadoop" >input/file1.txt

9. Create input directory on HDFS
   hadoop fs -mkdir -p input

10. Put input files to HDFS
    hdfs dfs -put ./input/* input

11. Run wordcount:
    hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar wordcount input output

12. Print the input files
    echo -e "\ninput file1.txt:"
    hdfs dfs -cat input/file1.txt

    echo -e "\ninput file2.txt:"
    hdfs dfs -cat input/file2.txt

13. Print the output of wordcount
    echo -e "\nwordcount output:"
    hdfs dfs -cat output/part-r-00000

14. To remove the docker containers using docker-compose: sudo docker-compose down

Alternatively, we can install the Hadoop Container using the following link: https://medium.com/@guillermovc/setting-up-hadoop-with-docker-and-using-mapreduce-framework-c1cd125d4f7b

1. Clone docker-hadoop repo from Github:
   https://github.com/big-data-europe/docker-hadoop

2. Start the necessary containers using docker-compose:
   docker-compose up -d

3. Get in our master node namenode:
   docker exec -it namenode bash

4. Visualise the dashboard in your localhost:
   Access to localhost:9870 in your browser

5. Copy the input file to the docker container:
   docker cp input.txt namenode:/tmp

6. Inside the namenode container, run the following hdfs commands
   hdfs dfs -mkdir -p /user/root
   hdfs dfs -mkdir /user/root/input
   hdfs dfs -put input.txt /user/root/input/

7. We need to write Map Reduce code in Java for a program (say. Word Count)
   Let us say the files are WordCountMapper.java, WordCountReducer.java and WordCountDriver.java.
   WordCountDriver.java has the main class (i.e. WordCountDriver). It will call the WordCountMapper and WordCountReducer classes.

8. Copy the java source files to the docker container:
   docker cp WordCountDriver.java namenode:/tmp
   docker cp WordCountMapper.java namenode:/tmp
   docker cp WordCountReducer.java namenode:/tmp

9. Inside the namenode container, compile the Java code:
   javac -classpath `hadoop classpath` -d . WordCountMapper.java WordCountReducer.java WordCountDriver.java

10. Inside the namenode container, create a Jar file:
    jar -cvf wordcount.jar *.class

11. Inside the namenode container, run the Hadoop job:
    hadoop jar wordcount.jar WordCountDriver /user/root/input /user/root/output

12. Check the output:
    hdfs dfs -cat /user/root/output/part-r-00000 > output.txt
    cat output.txt